

Technical Disclosure Commons

Defensive Publications Series

Architectural Design Generation Using Graph-Based Constraint Optimization and Conditioned Generative Models

Arpit Goyal

Follow this and additional works at: https://www.tdcommons.org/dpubs_series



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

Architectural Design Generation Using Graph-Based Constraint Optimization and Conditioned Generative Models

ABSTRACT

Generative models for architectural design can have difficulty adhering to complex constraints, such as directional rules from specific cultural frameworks, which may result in layouts with functional or symbolic limitations. A computational framework can address this by integrating rule-based logic with generative models. The system can translate user requirements and geospatial data into an abstract graph where rooms are represented as nodes. A logic engine, which can utilize a component such as a graph neural network, can optimize the placement of these nodes according to predefined adjacency and directional rules. The resulting optimized spatial graph can be used to create a conditioning mask that guides a generative model, such as a latent diffusion model, to render a floor plan. This approach can facilitate the creation of architectural designs that are more likely to comply with specified functional and symbolic orientation rules, potentially reducing a need for subsequent manual correction.

KEYWORDS

architectural design, generative models, constraint optimization, graph neural networks, latent diffusion models, floor plan generation, rule-based systems, spatial constraints, directional rules, adjacency requirements, generative artificial intelligence, geospatial orientation, design synthesis, automated design, computer-aided design

BACKGROUND

In the field of generative artificial intelligence for architectural design, some systems can have difficulty adhering to complex, non-linear constraints, for example, those dictated by cultural or prescriptive directional frameworks – examples include Vastu Shastra or Feng Shui. These models may produce layouts with functionally or directionally suboptimal room

placements. Such outputs can result in the need for manual correction, which can be costly and time-consuming. Some existing approaches may not fully address this issue. For instance, standard computer-aided design and building information modeling software may not include automated generation and can depend on manual compliance checking. Some generic floor plan generators may focus primarily on room adjacency while not fully considering certain cardinal directionality rules. Additionally, some specialized auditing applications may be configured to analyze existing designs rather than create new, optimized ones. There is no known existing mechanism that reconciles high-fidelity pixel generation with non-linear geometric rules like Vastu Shastra and GIS data. Consequently, a technical gap may exist for an automated mechanism that can reconcile design synthesis with the specific requirements of real-world geographic orientation and symbolic geometric rules during the generative process.

DETAILED DESCRIPTION

To address instances of generative models producing architecturally or directionally inconsistent layouts, a multi-tier computational framework for architectural design synthesis is described. The technology can integrate logical rule-based constraints with generative artificial intelligence models to facilitate the creation of compliant designs. This approach can help to reconcile the creative capabilities of generative models with functional and symbolic architectural constraints, such as those found in Vastu Shastra or Feng Shui, thereby reducing the need for manual correction.

An example technique involves a sequential process that begins by translating user inputs, such as plot dimensions and room requirements, into an abstract graph structure. The system proposed assumes that all information is rightfully obtained and that all appropriate permissions to use such information have been obtained. In this graph, for instance, rooms can be

represented as nodes and desired adjacencies as edges. A logic engine, which could be executed on a computing device (e.g., a server, desktop computer, or cloud computing platform) and may utilize a graph neural network, can optimize the placement of these nodes within the plot's boundaries. The optimization process can be guided by a loss function that considers both adjacency requirements and directional rules derived from a predefined ontology and geospatial data (e.g., true north orientation). The output of this stage may be a logically validated spatial graph that indicates a potential placement solution exists which satisfies the given constraints.

This validated spatial graph can then be used to generate a spatial layout mask, which may serve as a conditioning input for a generative model, such as a latent diffusion model. The mask can suggest the specific zones where each room or other architectural element could be rendered. By conditioning the generative process on this mask, the system may guide the model to produce a floor plan that is visually coherent and adheres to the pre-validated logical layout. The final output can include a 2D floor plan, a navigable 3D model, and a quantitative compliance report that provides a data-driven score assessing the design's adherence to the specified rules.

More particularly, with reference to FIGURE 1, the described systems and methods relate to a multi-tier computational framework for architectural design synthesis that can integrate logical rule-based constraints with generative artificial intelligence models. This framework can generate architectural layouts, such as floor plans, that adhere to specific directional and adjacency rules, which may be derived from established architectural or cultural principles. The system may operate through a sequential pipeline that can process user-defined inputs, apply logical optimization, for example, using graph-based neural networks, and render visual outputs using conditioned generative models. This approach may integrate generative models with

architectural constraints, which can automate certain aspects of a design process. The system may be architected in a modular fashion, and can comprise an input processing layer, a logic and optimization engine, a generative rendering pipeline, and an output reporting layer, which can interact to translate high-level requirements into compliant architectural designs.

The process can begin at a user input interface, which can be implemented as a component of a web-based application, a mobile application, or as a plugin or extension within existing professional architectural software suites. Examples of such suites include computer-aided design (CAD) or building information modeling (BIM) platforms. The user interface may be presented on various computing devices, such as a desktop computer, laptop, smartphone, smart watch, wearable device, or augmented reality glasses. Through this interface, a user may provide a set of initial parameters that define the design problem. These parameters can include the physical dimensions of the building plot, such as length and width, and a list of programmatic requirements for the structure. The requirements may detail the types of spaces or rooms desired (e.g., number of bedrooms, inclusion of a kitchen, a living area, bathrooms) and their approximate sizes or other properties. An input that may be captured through this interface is the geographic location of the plot. This can be accomplished by the user providing a street address, latitude and longitude coordinates, or by interacting with an integrated map component, for example, by placing a digital pin at a location on a map display. This geospatial information may be used for subsequent logic and orientation calculations. The user interface can gather information and forward it to a backend system for processing.

The data collected from the user interface may be transmitted to an input data processing application programming interface (API). This API can serve as an entry point for incoming design requests and can parse, validate, and structure the user-provided data. Upon receiving

geospatial coordinates, a function of this layer can be to communicate with an external or integrated geospatial information service capable of providing geographic data. This communication may be used to determine the True North orientation for the specified coordinates. True north, which is based on the Earth's axis of rotation, is distinct from magnetic north and can provide a fixed cardinal reference. This orientation may be useful for applying directional rules consistently. A call to the geospatial information service may return the angle of True North relative to the plot's assumed or specified orientation. The input data processing API can then collate the plot dimensions, programmatic requirements, and the calculated True North orientation into a structured data object that can be consumed by a subsequent stage of the pipeline, such as a logic engine.

Logical reasoning and spatial optimization can occur within a logic engine, which may be termed a spatial negotiator. This engine can operate on a formalized set of rules, which may be stored in an associated spatial database. The rule set may be structured as an ontology that digitizes the principles of a specific architectural or cultural framework. For example, this ontology may be represented as a data structure mapping programmatic spaces to their ideal locations within a directional schema. This schema could be conceptualized as a grid, for instance, a 9x9 grid representing the building footprint, which is then mapped to the eight cardinal and inter-cardinal directions (north, northeast, east, southeast, south, southwest, west, northwest) and a central zone. Each cell or region in this grid can be associated with a specific directional property. The ontology, therefore, might specify that a "kitchen" space has a high affinity for the "Southeast" zone and a "master bedroom" has an affinity for the "Southwest" zone. These rules, along with the True North orientation and plot data, may be stored and managed within a spatial database. The spatial database could be, for example, a relational

database with spatial extensions, allowing for storage and querying of georeferenced and geometric data. The ontology can be flexible, allowing for the substitution or modification of rule sets to accommodate different architectural styles or user-defined constraints.

To apply these rules, the logic engine can first translate the user's programmatic requirements into an abstract graph structure. In this graph-based representation, each room or architectural space (e.g., kitchen, bedroom 1, living room) can be modeled as a node. The relationships between these spaces can be represented as edges connecting the nodes. These relationships can encode different types of constraints, such as adjacency requirements. For example, if the design specifies for the dining room to be near the kitchen, an edge can be created between the dining room node and the kitchen node. The graph may thus become a topological representation of the desired layout, capturing functional adjacencies without yet defining precise geometric positions. This abstraction can allow the system to focus on satisfying relational constraints before committing to a physical layout. The strength of these adjacency requirements may also be weighted, indicating which connections are more beneficial than others.

Once the graph is constructed, the directional rules from the ontology can be encoded and assigned to the nodes. This may be accomplished by assigning directional weights to each node based on its type. These weights can function as a component of an objective or reward function that an optimization process may later use. For a given node, the weights can quantify the desirability of placing that node in various directional zones of the plot. For example, the kitchen node might be assigned a high positive weight for the Southeast zone and large negative weights for other zones. These weights may be stored as attributes of the nodes within the graph data structure. The combination of the graph's topology (nodes and edges) and the node attributes

(directional weights) can create a mathematical representation of the architectural design problem, encapsulating both adjacency and directional constraints in a single structure.

The optimization of this graph structure may be performed by a graph processing service, which can utilize a graph neural network (GNN). A GNN is a type of neural network that can operate on data structured as graphs. A mechanism of the GNN in this context can be an iterative process sometimes known as message passing or graph convolution. During this process, each node in the graph can repeatedly aggregate information from its neighboring nodes and update its own state. This state can include the node's current estimated position within a 2D coordinate system that corresponds to the building plot. Through multiple iterations of message passing, nodes may adjust their positions relative to their neighbors. A node connected by an edge may be pulled closer to its neighbor, attempting to satisfy the adjacency constraint. Simultaneously, each node may be influenced by its own directional weights, which can create a force pulling it towards its ideal cardinal zone on the plot.

The GNN may be trained to find an optimum for the positions of all nodes, seeking a state that minimizes a composite loss function. This loss function can be a mathematical expression that quantifies characteristics of a given layout. It can include a term for adjacency violations (e.g., the sum of distances between nodes that should be adjacent) and a term for directional violations (e.g., the sum of penalties for nodes being located outside their ideal directional zones). By iteratively adjusting node positions to minimize this loss, the GNN can converge on a set of optimized (x, y) coordinates for each node. The output of the graph processing service can be this set of coordinates, which represents an abstract but logically validated spatial layout. This output may be referred to as a validated spatial graph and can indicate that a placement solution exists that concurrently satisfies, to some degree, both the

adjacency and directional rules. This intermediate, validated artifact can be an output before proceeding to a more computationally intensive visual rendering stage.

A subsequent stage in the pipeline can be a generative AI pipeline, which can translate the abstract, validated graph into a visual architectural floor plan. This transition from the logical domain to the visual domain can be mediated by the creation of a spatial layout mask. The validated spatial graph, with its optimized node coordinates, may be used to generate this mask. The mask can be a 2D data structure, such as a bitmap image or a set of vector polygons, that delineates the intended zones for each room on the floor plan. For instance, based on the final coordinates and approximate size of the kitchen node, a corresponding polygonal area can be drawn on the mask. This may be repeated for all nodes, resulting in a mask that can resemble a simplified, labeled diagram of the floor plan. This mask can serve as a conditioning signal for the generative model, providing spatial guidance. It can indicate where each architectural element could be generated.

The generative pipeline may use a latent diffusion model (LDM), a type of deep generative model. An LDM can operate by starting with a tensor of random noise in a compressed latent space and iteratively refining this noise over a series of steps to produce a coherent image. The generation process can be guided by a text prompt, such as "a high-resolution architectural floor plan of a modern three-bedroom house." In this system, the LDM's generation may not be guided by text alone. It can also be guided by the spatial layout mask. This may be achieved through the use of a conditioning network, for example, a conditioning network architecture (such as ControlNet). This auxiliary network may run in parallel with the LDM and can be configured to accept conditioning inputs, such as the spatial layout mask. It can process the mask and inject this spatial information into various layers of the LDM during the

denoising process. This injection of information can guide the LDM to render pixels that align with the provided mask. For example, it may generate kitchen-like features within the area designated for the "kitchen" on the mask and bedroom-like features within the "bedroom" areas. This conditioning mechanism may improve the likelihood that the generated output aligns with the provided mask and the validated layout.

An output of the generative AI pipeline can be a high-resolution raster image of the floor plan. This image can then be passed to an output and reporting layer for presentation to the user. This layer can assemble and deliver the set of design artifacts. In addition to the 2D floor plan image, this layer can generate a 3D rendering of the design. Using the 2d layout data and assuming standard wall heights, it can construct a simple 3d model of the space. This model can then be rendered using a client-side library, for example, a client-side 3D graphics library, allowing the user to experience the design through an interactive, navigable 3D walkthrough. This may provide a more intuitive understanding of the spatial qualities of the generated layout. The communication of these results can be facilitated via a web-based dashboard or directly within the user's host CAD or BIM application. To facilitate interoperability with professional industry standards, the framework implements a RESTful API (specifically utilizing FastAPI) that enables bidirectional communication between the central logic engine and third-party architectural software suites.

In addition to from-scratch design synthesis, the framework includes a Remedy Engine configured as an image-to-image variant for auditing and correcting existing architectural layouts. The system can ingest an existing, non-compliant floor plan as a raster or vector input to evaluate its adherence to directional rules. Using a proprietary reward function, the engine calculates a score based on the proximity of room centroids to their ideal directional zones. For

designs identified as non-compliant, the engine suggests structural "fixes" by inpainting new, optimized layouts over the original plan. The generative process maintains the overall building footprint and fixed structural elements while reconfiguring internal partitions to satisfy the logical validated graph.

CONCLUSION

Systems and methods are described for generating architectural designs that consider specific directional constraints. A system may utilize a graph-based optimization engine to compute a spatial layout that accounts for both adjacency and directional rules, which can be informed by geospatial data and a defined ontology. This optimized layout can then be used to generate a conditioning input, such as a spatial mask, for a generative artificial intelligence model. This process may guide the generative model to render floor plans that are more likely to be compliant with the design rules, potentially mitigating instances where unconstrained systems could produce functionally incorrect layouts.

FIGURE 1

